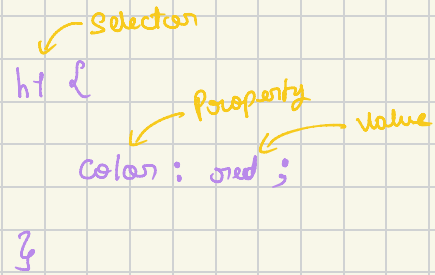
CSS (cascading style sheet)

- It is a language that is used to describe the style of a document.

Basic Syntax



Including Style

Inline –

```
<h1 style = " color : red " > Apna College </h1>
```

<Style> tag – We generally follow this

```
<style>
    h1 {
        color : red ;
    }
</style>
```

Including Style

External Stylesheet – Writing CSS in a separate document & linking it with HTML file

Color Property
used to set the color of foreground

color : red ;
color : pink ;
color : blue ;
color : green ;

Background colour property - used to set the color of background

background-color : red ;
— u — : pink ;
— u — : blue ;
— u — : green ;

HTML

<body>
<h1> This is a heading </h1>
 This is a simple & paragraph
</body>

CSS

Selector

body {

background-color : aqua ;

}

Selectors

① universal Selector

*{}

② Element Selector

h1{}

③ Id Selector

```
#myId {}
```

④ Class Selector — it's used to give some color or style to a multiple elements

```
.myClass {}
```

* Do not use n.o in id
- like id="101"
  And then #101

- CSS ignore this in Her value or invalid Syntax, causing the entire CSS rule to be ignored.

- best practice

```
<h1 id="heading-1"> CSS file here </h1>

#heading-1 {

    color: greenyellow;

}
```

* use classes for Stylings
* use IDs for specific Anchors & Scripts
* Neither IDs nor classes should start with n.o

```
h1, h2, h3 {
    color : red ;
}
```

this will set the color to red to all h1, h2, h3

Text Properties

① Text - alignment

text-align : left / right / center

② Text - decoration

text - decoration : underline / overline / line-through

- It is generally used to remove underline

Just set -

text-decoration : none ;

- for setting color

text-decoration : blue wavy underline ;

② font - weight

font-weight : normal / bold / bolder / lighter

font-weight : 100 - 900

③ font - family

font-family : arial

font-family : arial, sub-category

- we also use multiple font-family because different browsers support different language
- By selecting multiple it work from left to right

eg arial, roboto, Time-roman
- arial → If it's doesn't work
- roboto → Then this one
- Time-roman → And this one

units in CSS

Absolute

pixels (px)

96px = 1inch

font-size : 2px ; font-size : 10px ;

④ line-height

line-height : 2px — distance b/w a line

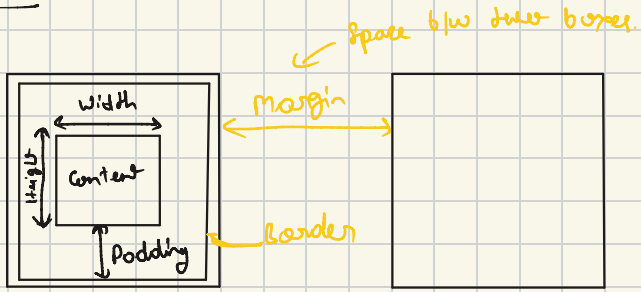
line-height : normal

⑤ text-transform

text-transform : uppercase / lowercase / capitalize / none

Box Model in CSS

① Height
② Width
③ Border
④ Padding
⑤ Margin



① Height – By default, it sets the content area height of the element

```
div {
    height : 50px ;
}
```

② Width – By default, it sets the content area width of the element

```
div {
    width : 50px ;
}
```

③ Border – used to set an element's border

```
border-width : 2px ;
border-style : solid / dotted / dashed
border-color : black ;
```

★ Border – Shorthand

```
border : 2px solid black ;
```

- Used to round the corners of an element's outer border edge

```
border-radius : 10px ;
border-radius : 50% ;
```

④ Padding

- Padding-left
- Padding-right
- Padding-top
- Padding-bottom

```
Padding-left : 20px ;
Padding-right : 20px ;
Padding-top : 20px ;
Padding-bottom : 20px ;
```

★ shorthand - padding

```
Padding : 1px 2px 3px 4px    (clock wise)
          ↓   ↓   ↓   ↓
          top right bottom left
```

```
Padding : 20px   - It will for all sides with 20px
```

⑤ Margin - area b/w two boxes

```
margin-right : 20px ;
margin-left : 20px ;
margin-top : 20px ;
margin-bottom : 20px ;
```

Shorthand

```
margin : 50px ;
margin : 1px 2px 3px 4px ;
         top right bottom left  (clock-wise)
```

Display Property

inline – Take only the space required by the element. (no margin / padding)

block – Take full space available in width.

Inline-block – Similar to inline but we can set margin & padding

none – To remove element from document flow.

```
display : inline / block / inline-block / none
```

Visibility

```
visibility : hidden ;
```

Note : when visibility is set to none, space for the element is reserved.
But for display set to none, no space is reserved or blocked for the element.

Alpha channel – opacity (0 to 1)

- RGBA
  Red green blue Alpha

```
color : rgba ( 255, 0, 0, 1) ; – full visibility
```

Unit in CSS

① Relative

Percentage (%) — It is often define a size as relative to an element's parent object.

```
width : 33.0 ;
margin-left : 50% ;
```

eg — `<div>` — Parent
`<div>` — child
`</div>`
`</div>`

— If we set width : 33%, according to parent `<div>`

Em — Relative to font size of the parent, in the case of typographical properties like font-size, and font size of the element itself, in the case of other properties like width.

HTML

```
<body>
  <div id="Box1">
    Box1
    <div id="Box2">
      Box2
    </div>
  </div>
</body>
```

```
Box1 {
  background-color : yellowgreen ;
  height : 100px ;
  width : 100px ;
  font-size : 10px ;
}
Box2 {
  background-color : yellow ;
  height : 50px ;
  width : 50% ;
  margin-left : 10% ;
  font-size : 2em ;
}
```

2 × (10px) (parent)

width : 5em ;

↳ 5px (2 × 10px)
(font-size of element itself)

Rem (Root Em) – font size of the root element.

font-size : 2rem ;
↳ (2 × 16px)
from root element
eg – <body> ← root element
Box

</body>

Others

vh : relative to 1% viewport height
height : 1vh (1% height of screen)
vw : relative to 1% viewport width
width : 1vw (1% width of screen)

Position

The position CSS property sets how an element is positioned in a document.

position : static / relative / absolute / fixed

① static : default position (The top, right, bottom, left, and z-index properties have no effect)

position: static;

② relative : element is relative to itself (The top, right, bottom, left, and z-index will work)

position : relative ;
top : 10px ; → shift 10px down

left : 10px ; → Shift 10px to right

② absolute : Positioned relative to its closest positioned ancestor. (removed from the flow)

positioned
non-static

```
position : absolute ;
top : 10px ;
left : 10px ;
```
— It will set 10px from top of screen (body)
10px from left.

⑨ fixed : Positioned relative to browser. (removed from flow)

```
position : fixed ;
right : 10px ;
```
— 10px from right of the browser/document

```
#box4 {
    position : fixed ;
    top : 0 ;
}
```
It will fixed the position of the box of the top even if we scroll down it will not move.

⑤ sticky : Positioned based on user's scroll position

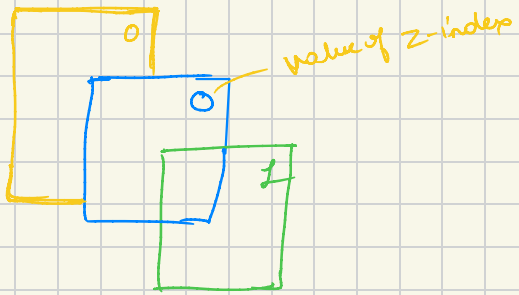
```
#box3 {
    position : sticky ;
    top : 10px ;
}
```
It will stick the position of the box from top : 10px, when it reach it position while scrolling.

z-index — It decides the stack level of elements

overlapping elements with a larger z-index cover those with a smaller one.

```
z-index : auto (0)
z-index : 1 / 2 / ...
z-index : -1 / -2 / ...
```
— +ve coming upward
— -ve value going downward

eg -



Background Image - used to set an image as background

```
background-image : url("image.jpeg");
```

Background Size

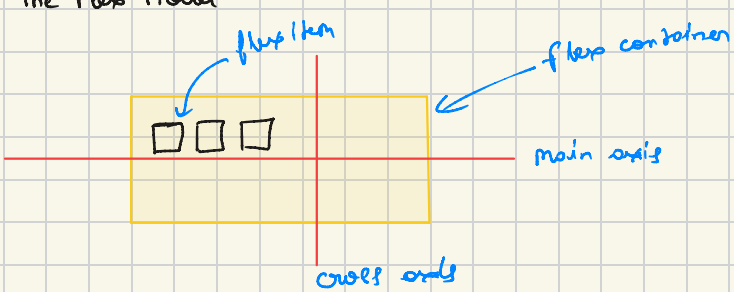
```
background-size : cover / contain / auto
```

- We can not use background-image alone, we have to use background-size with it to fit it accordingly.
- By default it is auto.

Flexbox

Flexible Box Layout - It is a one dimentional method for arranging items in rows or columns.

The flex Model



- main axis can be cross axis or visa versa but both always in 90° of each other.

Flex Direction

It sets how flex items are placed in the flex container, along which axis and direction.

fixed set –

Display : flex ;

flex-direction : row → main axis left to Right / row-reverse / column / column-reverse

↳ eg - Box 5, 4, 3, 2, 1

- when we change to column, main axis changes to vertical.
- if the box size in px is more than the height of the flex box then it shrinks it won't if allowed by the content.

Flex Properties – for flex container

① Justify-content — alignment along the main axis.

justify-content : flex-start ; / flex-end / centre / space-around / space-evenly ;

↳ It will set the start (←→)

↳ start at end

but it will not change the order

eg – Box-1, 2, 3, 4, 5

justify-content : space-around / space-between ;

↳ box will be placed evenly in space in row.

↳ It will left no space at start and end.

② flex-wrap — nowrap / wrap / wrap-reverse

flex-wrap : nowrap / wrap / wrap-reverse ;

③ align-items — alignment along the cross axis.

align-items : flex-end / center / flex-start ;

④ align-content — alignment of space b/w & around the content along cross-axis

align-content : flex-start / flex-end / center ;

flex property — for flex item

① align-self - alignment of individual along the cross axis.

align-self : flex-end / flex-start / center

② flex-grow - how much a flex item will grow relative to the rest of flex items if space is available

flex-grow : 0 / 1 / 2 / 4 — ;
- → grow at the power of 2
- ↳ it will not grow
- ↳ grow at the power of 1

① flex-shrink — how much a flex item will shrink relative to the rest of flex items if space is available.

flex-shrink : 0 / 1 / 2 / 3 — ;
- ↳ it will not shrink

@ Which has higher priority - align-items or align-self ?
- align-items used for flex containers
- align-self used for individual - it has high priority

Media queries - Help to create a responsive website

```
@media (width : 600Px) {
    div {
        background-color : red ;
    }
}

@ media (min-width : 600Px) {
    div {
        background-color : red ;
    }
}
```

Combined media Queries :

```
@media (min-width : 200 Px) and (max-width : 300Px) {
    div {
        background-color : red ;
    }
}
```

Transitions – Transitions enable you to define the transition b/w two states of an element.

```
div {
    height : 100px ;
    width : 100px ;
    background-color : blue ;
    border : 2px solid black ;
}

div:hover {        (Pseudo-classes)
    background-color : red ;
}
```

Pseudo-Class

:hover
:active
:focus
:focus-within
:target
:visited
:focus-visible

transition-property — Property you want to transition (font-size, width etc.)
transition-property : all ;

transition-duration — 2s / 4ms ....
transition-duration : 2s / 4ms ..
transition-timing-function — ease-in / ease-out / linear / steps ....
transition-timing-function : ease-in / ease-out / linear / steps ..
transition-delay — 2s / 4ms ....
transition-delay : 2s / 4ms ...

duration delay

transition : font-size 2s ease-in-out 0.2s ;

property name — timing-function

CSS Transform — used to apply 2D & 3D transformations to an element

① rotate

```
transform : rotate(45deg) ;
```

```
rotate : 45deg ;
rotateX : 45deg ;
rotateY : 45deg ;
rotateZ : 45deg ;
```

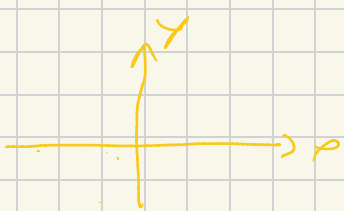
} generally used for 3-D

② Scale

```
transform : scale(2) ;   → In both X, Y
transform : scale(0.5) ;
transform : scale(1,2) ;
                  x-axis  y-axis

transform : scaleX(0.5) ;
transform : scaleY(0.5) ;
```

③ translate




```
transform : translate(20px) ;
transform : translate(20px, 50px) ;
```

```
transform : translateX(20px) ;  → move forward +ve x-axis
transform : translateY(20px) ;  → move forward +ve Y-axis
            Y(-20px) ;          → -ve Y-axis
```

④ Skew — it Skew an element on the 2D plane.

Animation — To animate CSS elements

Animation name

```
@keyframe myName {
    from { font-size : 20px ; }
    to { font-size : 40px ; }
}
```

Animation Properties

animation-name

name

① animation-name : colorAnimation ;

② animation-duration : 2s ; — how long

③ animation-timing-function : ease-in / ease-out / ease-in-out

④ animation-iteration-count : 5 ;

⑤ animation-direction : normal / reverse / alternate / alternate-reverse ;

normal — cycle
reverse — to-from
alternate — to-from, from-to

```
@keyframes animation {
    from { left : 0px; }
    to { left : 200px; }
}
```

timing-function
iteration-count

animation : myName 2s linear 3s infinite normal

myName — name
2s — duration
3s — delay
infinite — iteration-count
normal — direction

o/o in Animation

```
@keyframe myname {
    0% { font-size : 20px ; }
    50% { font-size : 30px ; }
    100% { font-size : 40px ; }
}
```

eg

```
@keyframe colorAnimate {
    0% {
        left : 0px ;
        background-color : yellow ;
    }
    50% {
        left : 100 px ;
        background-color : blue ;
    }
    100% {
        left : 200 px ;
        background-color : green ;
    }
```

Practice Set-8

Q Create a simple loader using CSS

Step 1 — Create a div with circular shape & a thick border from one end (top / bottom / left / right)

Step 2 — To make it spin create an animation which transforms it from 0deg to 360deg

Step 3 — Add the animation property to the loader with infinite duration

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Powdice-ps</title>
    <link rel="stylesheet" href="ppstyle.css">
</head>
<body>
    <div class="head"></div>
</body>
</html>
```

CSS

```
div {
    height: 200px;
    width: 200px;
    border-radius: 50%;
    border-left: 20px solid black;
    animation: spinAnimation 3s steps(5) 0s infinite normal;
}

@keyframes spinAnimation {
    from {
        transform: rotate(0deg);
    }
    to {
        transform: rotate(360deg);
    }
}
```